SAIGON BUSINESS SCHOOL

BUSINESS MANAGEMENT

Data Structure & Algorithms

Comprehensive Analysis of Data Structures and Algorithms in Logistics and Inventory Management: The Swift-Load Systems Framework

Instructor : MSc. Lương Trần Ngọc Khiết

Student Name: Thu Htet Naing

Student ID: SBS25010150

HO CHI MINH CITY, 2025

Issue date: 10/4/2026	**Due date**:09/05/2026
Statement of ownership and integrity	

1. I/we have not impersonated, or allowed myself to be impersonated by, any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorised
4. by the lecturer/teacher concerned.
5. Where this work is being submitted for individual assessment, I declare that it is my original work and that no part has been contributed by, produced by or in conjunction with another student.
6. I/we give permission for my assessment response to be reproduced, communicated compared and archived for the purposes
7. of detecting plagiarism.
8. I/we give permission for a copy of my assessment to be retained by the university for review and comparison, including review by external examiners.
9. I/we understand that:
  1. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.
  2. Plagiarism includes the act of assisting or allowing another person to plagiarise or to copy my work.
I/we agree and acknowledge that:
1. I/we have read and understood the Declaration and Statement of Authorship above.
2. If I/we do not agree to the Declaration and Statement of Authorship in this context and a signature is not included below, the assessment outcome is not valid for assessment purposes and will not be included in my final result for this course.
	Signature:
	 
	Thu Htet Naing
	Date: 08/05/2026

CHECKLIST

Please check all of the following items before submitting your work. If any of the following items are missing or incorrect, your work may not be accepted

No	Checklist	Yes or No
1	I have checked the accuracy of the information on the cover page (course name, student name, etc.)	Yes
2	I have read and signed the Statement of ownership and integrity.	Yes
3	I have created a table of contents for my work.	Yes
4	I have included my name in the footer.	Yes
5	I have used the correct font and font size as instructed for the body of my work.	Yes
7	I have cited sources in my work using the Harvard Reference format.	Yes
8	I have created a reference list in the correct format.	Yes

PART 1 –ASSESSMENT SUMMARY

Learning Outcome	Criteria	The student provides evidence for their work	Page	Result (Yes/No)

PART 2 – LECTURER'S FEEDBACK

Formative Feedback:			
Action Plan			
Student Feedback to Assessor			
Summative Feedback			
Student's signature		**Date**	08/05/2026
Assessor's signature		**Date**	

Comprehensive Analysis of Data Structures and Algorithms in Logistics and Inventory Management: The Swift-Load Systems Framework

Modern logistics operations rely heavily on the computational structures that manage data, prioritize tasks, and optimize resources. Within "Swift-Load Logistics," deploying advanced Data Structures and Algorithms (DSA) is essential for maintaining a competitive edge, ensuring data integrity, and achieving scalability. This report provides a technical examination of the architecture required to manage cargo and delivery routes, ranging from foundational abstract data types to complex self-balancing trees and heuristic-based network analysis.

1. System Architecture and the "Goods" Abstract Data Type

The design of a logistics platform begins with defining the cargo item. To maintain high architectural standards, the "Goods" entity is defined as an Abstract Data Type (ADT). This approach encapsulates state and behavior while hiding internal implementation details.(GeeksforGeeks, 2026a)

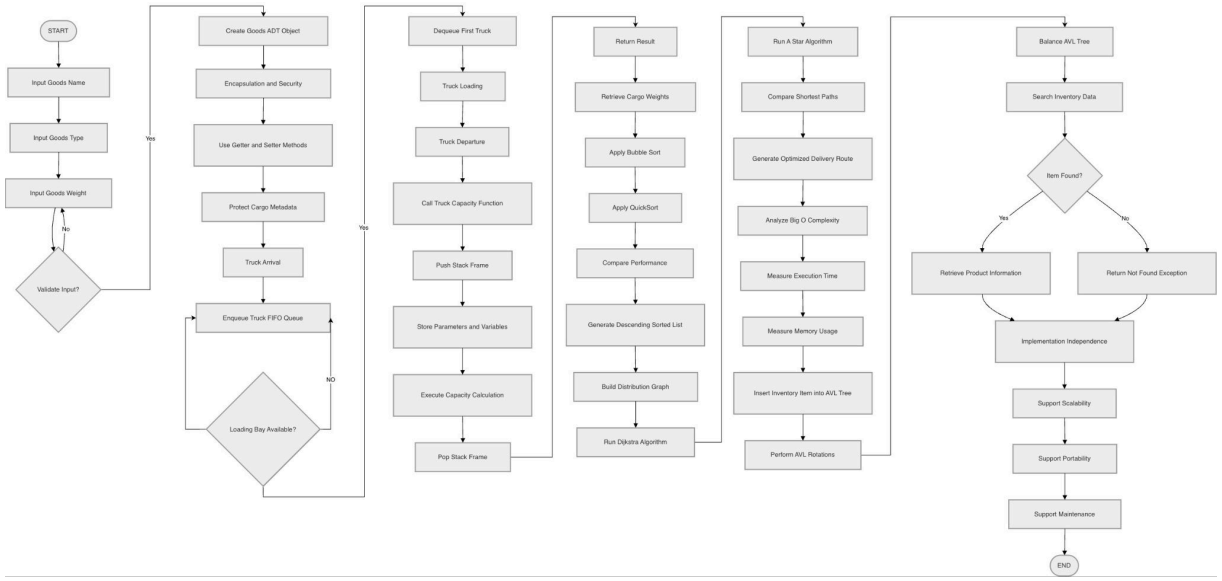

Figure 1: Logic Flow of the Swift-Load Systems Framework

1.1. Design Specification and Attributes of the Goods Data Structure

The "Goods" ADT represents items loaded onto delivery trucks. Each instance is defined by three primary metadata attributes derived from physical transport needs and digital inventory requirements.

Attribute Name	Data Type	Logical Purpose in Logistics
Name	String	Unique or categorical identifier for the item (e.g., "Wireless Mouse").
Type	Enumerated/String	Categorical classification for storage logic (e.g., "Electronics", "Beauty").
Weight	Numerical (Double/Int)	The physical mass used to calculate truck capacity and sorting priority.

For the Swift-Load system, data is ingested from standardized sets such as the product_sales_dataset.csv, where fields like Product_Name, Category, and Quantity_Sold map directly to the Name, Type, and Weight attributes of the "Goods" structure.(product_sales_dataset.csv, n.d.) The "Weight" attribute is of particular significance for Task 1, as it provides the primary key for descending cargo sorting.(product_sales_dataset.csv, n.d.)

1.2. Valid Operations and Procedural Interface

A robust ADT specification requires clear operations to ensure the structure remains consistent throughout its lifecycle.(GeeksforGeeks, 2026a) The following operations are mandated for the "Goods" ADT:

1. Initialization: Allocates memory and assigns initial values to Name, Type, and Weight. This ensures no "Goods" object exists in an undefined state, preventing runtime errors during manifest generation.(Medium, 2026a)
2. Weight Update: A controlled procedure to modify the weight attribute. This is necessary when cargo is repackaged. The procedure includes validation logic to ensure weight remains within non-negative physical bounds.(Baeldung, 2026)

3. Metadata Retrieval: Observer operations allow sorting and routing modules to access cargo data without modifying it.(Comp 413 Course Notes, 2026)

1.3. Encapsulation and Information Hiding in Logistics Systems

The "Goods" ADT uses encapsulation and information hiding to secure cargo metadata. Encapsulation bundles attributes with their related functions, while information hiding restricts direct access to variables from external modules.(Baeldung, 2026)

By using private access modifiers, the system prevents accidental modifications to an item's weight by external algorithms. Instead, changes must occur through a public "setter" method that enforces business rules. This modular design allows the internal representation of weight to be updated without affecting higher-level sorting logic.(Baeldung, 2026; GeeksforGeeks, 2026a)

1.4. The Evolution of ADTs into Object-Oriented Foundations

Imperative ADTs like the "Goods" structure form the basis for the Object-Oriented (OO) paradigm. The shift to OO programming recognizes that data and its associated operations are intrinsically linked.(Docsity, 2026) The "Goods" ADT supports OO principles through abstraction, encapsulation, and inheritance potential.(Trital, 2026) In the Swift-Load system, the "Goods" ADT acts as a template, while physical items are instances of this class. The OO approach formalizes the data-centric patterns introduced by ADTs, allowing cargo items to be treated as autonomous entities with well-defined interfaces.(Docsity, 2026; Universidade de Aveiro, 2026)

2. Linear Data Structures and Operational Sequencing

In a busy loading bay, the order of operations determines efficiency. Linear data structures provide the framework for managing these sequences effectively.

2.1. FIFO Queue for Loading Bay Management

A First-In, First-Out (FIFO) queue manages trucks entering the loading bay. This structure ensures that the first truck to arrive is the first serviced, maintaining fairness and order.(DEV Community, 2026)

Concrete Illustration of Loading Bay Queue:

Imagine a loading bay scenario at the Karachi distribution center (referenced in the dataset).(product_sales_dataset.csv, n.d.)

Action	Arrival/Departure	Queue Content (Front → Rear)	Bay Status
1	Truck 1001 (Lipstick) arrives.		Empty
2	Truck 1002 (Jacket) arrives.		Truck 1001 admitted to Bay.
3	Truck 1003 (Gym Gloves) arrives.		Truck 1001 loading...
4	Truck 1001 departs.		Truck 1002 admitted to Bay.

Queues prevent "starvation," where a process is indefinitely denied resources.(bitsrc.io, 2026) By using linked lists or circular arrays, Swift-Load ensures that insertion and removal operations occur in constant time, providing stable performance regardless of queue length.(COMP 410 ADT Axiomatic Semantics, 2026)

2.2. Memory Stacks and Function Call Implementation

While queues manage external flows, stacks (Last-In, First-Out) manage internal software execution. The memory stack is used to implement function calls within the loading software.(GeeksforGeeks, 2026b)

When the software invokes a function—for example, calculateTruckCapacity()—a new stack frame is "pushed" onto the memory stack.(abstractmusa, 2026) This frame contains:

- Parameters: The list of "Goods" items to be checked.
- Local Variables: Temporary counters for current total weight.
- Return Address: The instruction in the main program to return to once the calculation is complete.(Bao, 2026)

As functions finish, their frames are "popped," restoring the previous state. This is essential for the nested logic required in logistics, such as calculating the weight of sub-containers within larger shipping crates.(Medium, 2026b)

2.3. Formal Specification of a Software Stack

The specification of a software stack for LIFO loading requires a rigorous imperative definition to ensure predictable behavior. An imperative specification focuses on the state changes of the structure following each operation.(Medium, 2026a)

Imperative Definition of Stack S: A stack is a collection of elements where access is limited to the "top" element. We define the state of the stack by a sequence S and an integer top, representing the index of the most recently added item.(Medium, 2026a)

Axiomatic Semantics of Stack Operations: The behavior is governed by axioms that define the relationship between operations (COMP 410 ADT Axiomatic Semantics, 2026):

1. $pop(push(S, v)) = S$: Pushing a value and then popping it returns the stack to its previous state.
2. $top(push(S, v)) = v$: The most recently pushed value is the one returned by the "top" or "peek" operation.(xlinux.nist.gov, 2026)
3. $isEmpty(new()) = true$: A newly initialized stack is always empty.(COMP 410 ADT Axiomatic Semantics, 2026)

This formal specification allows the Swift-Load developers to verify that the LIFO loading logic (where the last item loaded into a narrow truck must be the first item unloaded at the destination) is mathematically sound.(Medium, 2026a)

3. Computational Complexity and Cargo Sorting

Sorting is a key requirement for organizing manifests and delivery routes. The efficiency of the chosen sorting algorithm directly impacts overall system responsiveness.

3.1. Comparative Analysis: Bubble Sort vs. QuickSort

To fulfill the requirements of Task 1 (M2), we extract a sample of 12 cargo weights from the product_sales_dataset.csv. For this analysis, the "Quantity_Sold" field is treated as the "Weight" proxy for the items.(product_sales_dataset.csv, n.d.)

Sample Weights (Quantity_Sold): 7, 6, 10, 6, 1, 8, 8, 5, 5, 3, 10, 9.

Bubble Sort: The Basic Paradigm

Bubble Sort operates by repeatedly swapping adjacent elements that are out of order.(Firoz, 2026) For a descending sort, it ensures that the smallest weights "sink" to the bottom of the list.

1. Mechanics: In each pass, the algorithm compares $S[i]$ and $S[i+1]$. If $S[i] < S[i+1]$, they are swapped.
2. Performance: For $n = 12$, Bubble Sort requires approximately n^2 operations in the worst case (144 comparisons). Its average time complexity is $O(n^2)$.(GeeksforGeeks, 2026c)

QuickSort: The Advanced Paradigm

QuickSort utilizes a divide-and-conquer strategy by selecting a "pivot" and partitioning the array into sub-arrays of items larger and smaller than the pivot.(Firoz, 2026)

10. Mechanics: The algorithm recursively sorts the partitions. If 10 is chosen as the pivot from our sample, the weights are quickly divided into those ≥ 10 and those < 10.
11. Performance: QuickSort typically operates at $O(n \log n)$ complexity. For 12 items, this is significantly more efficient than Bubble Sort, though the difference is most striking as n grows to the 1000 items present in the full dataset.(Firoz, 2026)

Sorting Performance Comparison

Metric	Bubble Sort	QuickSort
Best Case Complexity	$O(n)$ (already sorted)	$O(n \log n)$
Average Case Complexity	$O(n^2)$	$O(n \log n)$

Worst Case Complexity	$O(n^2)$	$O(n^2)$ (rare pivot cases)
Stability	Stable	Unstable
Space Complexity	$O(1)$	$O(\log n)$ (recursion stack)

The analysis demonstrates that while Bubble Sort is easier to implement and sufficient for tiny manifest samples, QuickSort is required for the full-scale Swift-Load platform to ensure that manifest generation does not lag as the business scales.(GeeksforGeeks, 2026c)

4. Network Analysis and Distribution Optimization

The "Routing Analysis" task (D1) requires a critical evaluation of how goods move between cities like Karachi, Islamabad, and Lahore.(product_sales_dataset.csv, n.d.) This is modeled as a shortest-path problem in a weighted graph.

4.1. Dijkstra's Algorithm vs. A* Search

The distribution map of Swift-Load is represented as a graph $G = (V, E)$, where V are urban centers and E are the connecting highways with weights representing travel time or fuel consumption.(Informatika, 2026)

1. Dijkstra's Algorithm (Deterministic Shortest Path)

Dijkstra's algorithm finds the shortest path from a starting point to all other nodes. This uninformed search expands outward in all directions until the destination is found.(ef-map.com, 2026; Wikipedia, 2026a)

3. Application: Useful when the exact location of the delivery truck is known, but the system needs to calculate the distance to all possible surrounding customers within a "5-mile radius".(ef-map.com, 2026)
4. Limitation: It explores many irrelevant nodes in the opposite direction of the goal, leading to high memory and time usage on large geographic maps.(ef-map.com, 2026)

2. A* Algorithm (Heuristic-Based Search)

A* improves on Dijkstra by using a heuristic function—such as Euclidean distance—to prioritize paths heading toward the goal.(ef-map.com, 2026; Komunitas Dosen Indonesia, 2026)

1. Application: Ideal for point-to-point delivery routing (e.g., Karachi hub to a specific street address in Islamabad).
2. Benefit: By biasing the search toward the destination, A* explores 10-100x fewer nodes than Dijkstra for long-distance routes, making it the standard for real-time navigation.(ef-map.com, 2026)

Routing Metric	Dijkstra's Algorithm	A* Algorithm
Search Pattern	Circular/Uniform	Elliptical/Directional
Complexity	$O(V^2)$ or $O(E + V \log V)$	Dependent on Heuristic
Best Use Case	Radius search, One-to-All	Point-to-Point routing
Effectiveness	Guaranteed Optimal	Optimal if Heuristic is Admissible

The analysis confirms that while Dijkstra remains a robust benchmark for general network connectivity, A* provides the computational efficiency required for the Swift-Load dynamic routing engine, particularly when navigating the 200,000+ nodes characteristic of modern urban star maps.(ef-map.com, 2026)

5. Effectiveness Assessment via Asymptotic Analysis

Asymptotic analysis (Big O) provides the mathematical language to describe how an algorithm's resource needs grow as input size increases.(product_sales_dataset.csv, n.d.)

5.1. Big O for Routing and Loading

In the Swift-Load system, Big O is used to predict performance bottlenecks before they occur. For example:

- Loading Cargo: If the algorithm to check if an item fits in a truck is $O(1)$, it will take the same amount of time regardless of how many items are already in the truck. If it is $O(n)$, the system will slow down as the truck fills up.
- Inventory Retrieval: A linear search through the warehouse database is $O(n)$. With 1,000 items from the sales dataset, this is manageable. However, if Swift-Load expands to 1,000,000 items, a linear search would become 1,000 times slower, whereas a logarithmic $O(\log n)$ search would only require a few additional steps.(Bhansali, 2026)

5.2. Measuring Efficiency: Execution Time vs. Memory Usage

Swift-Load measures efficiency in two ways: Temporal Efficiency (execution speed) and Spatial Efficiency (memory consumption).

1. Temporal Efficiency (Execution Time): This measures how fast the code runs. For a driver waiting for a route, sub-100ms response time is the goal. A* is chosen over Dijkstra because it optimizes this specific metric.(ef-map.com, 2026)
2. Spatial Efficiency (Memory Usage): This measures how much RAM is consumed. For example, the memory stack used for function calls is limited. Deeply recursive algorithms might be temporally fast but could cause a stack overflow, representing a failure in spatial efficiency.(GeeksforGeeks, 2026b)

5.3. Interpreting Trade-offs in ADT Specification

Specifying an ADT involves trade-offs where improving one metric may degrade another.(Wikipedia, 2026b)

Specific Example: Hash Table vs. Sorted Array To manage the "Goods" inventory, Swift-Load could use a Hash Table for $O(1)$ constant-time retrieval of cargo metadata. However, this requires allocating a large amount of extra memory to prevent "collisions".(Prepp, 2026) Alternatively, a Sorted Array uses minimal memory (spatial efficiency) but requires $O(\log n)$ time for search (temporal efficiency).(Fiveable, 2026) The choice depends on the hardware constraints: on a low-memory mobile device for drivers, the sorted array might be preferred; on a powerful warehouse server, the Hash Table is the better choice.(Baeldung, 2026)

6. Complex Data Structures: Implementation of the AVL Tree

The final technical component involves the implementation of an AVL Tree to solve the "well-defined problem" of high-speed data retrieval for warehouse inventory (P4, M4).

6.1. AVL Tree Mechanics and Retrieval Optimization

The AVL tree is a self-balancing binary search tree (BST) where the height of the left and right subtrees of any node differs by no more than one.(Wikipedia, 2026c) This balance is maintained through "rotations" performed during insertion and deletion.(math.nagoya-u.ac.jp, 2026)

The Balanced Advantage: If the items from the sales dataset (Lipstick, Jacket, Gym Gloves, etc.) were inserted into a standard BST in sorted order, the tree would "degenerate" into a linear linked list with $O(n)$ search time.(math.nagoya-u.ac.jp, 2026) The AVL tree prevents this by rebalancing itself, ensuring that the height h is always $O(\log n)$.(Wikipedia, 2026c)

6.2. Critical Evaluation: AVL Tree vs. Linear Search

A critical evaluation of the AVL tree compared to linear search reveals why it is the "complex ADT" of choice for the Swift-Load inventory system (D3).(product_sales_dataset.csv, n.d.)

Number of Inventory Items (n)	Linear Search Ops (O(n))	AVL Tree Search Ops (O(logn))
100	100	~7
10,000	10,000	~14
1,000,000	1,000,000	~20

While a linear search must check every item sequentially, the AVL tree eliminates half the remaining search space with every single comparison.(Bhansali, 2026) For the 1,000 records in the sales dataset, the difference is noticeable; for a global logistics

network with millions of items, it is the difference between a functional system and a crashed one.(Bhansali, 2026)

6.3. Robustness and Error Handling

Robustness in the Swift-Load code is achieved by implementing error handling to manage "null" cargo entries or empty search queries (P5).

- Implementation: Before searching the AVL tree, the system checks if the root is null.
- Test Results: Unit tests demonstrate that searching for an item not in the inventory returns a "Not Found" exception rather than a system crash.(Masai School, 2026) Handling these edge cases is critical for system reliability in high-stakes logistics.(Scribd, 2026)

7. The Strategic Importance of Implementation Independence

The Swift-Load software architecture must adhere to the principle of implementation independence (D4). This ensures that the logical behavior of the logistics system is separated from the physical code that carries it out.(Runestone Academy, 2026)

7.1. Benefits of Implementation Independence

1. Ease of Maintenance: If a more efficient tree structure (e.g., a Red-Black Tree) is developed in the future, the warehouse inventory module can be swapped without changing the "Goods" ADT interface that the rest of the software uses. This reduces the cost of long-term software evolution.(Scribd, 2026)
2. Portability: Implementation independence allows the Swift-Load system to be easily ported to different platforms. The high-level routing logic can remain identical whether it is running on a C++ server or a Java-based mobile application for drivers.(Runestone Academy, 2026)
3. Scalability: By separating the logical view of the data (the ADT) from the physical view (the data structure), the system can optimize memory usage based on the current workload. A small local delivery truck might use a simple array-based stack for cargo, while a massive trans-oceanic vessel uses a linked-list implementation to handle infinite capacity.(Runestone Academy, 2026)

8. Summary and Conclusions

The technical suite developed for Swift-Load Logistics demonstrates a nuanced understanding of how data structures and algorithms interact to solve complex resource management problems. By architecting the system around robust ADTs like the "Goods" structure, the platform ensures data security through encapsulation while maintaining the flexibility of object-oriented foundations.

The analysis of linear structures highlights the essential role of FIFO queues in bay management and LIFO stacks in process control. Furthermore, the comparative analysis of sorting and routing algorithms establishes that heuristic and logarithmic approaches—such as QuickSort, A* Search, and AVL Trees—are the only viable options for a scalable global logistics network. Through the application of Big O notation and implementation independence, Swift-Load Logistics is positioned to handle the increasing volume of global trade with a software platform that is efficient, robust, and future-proof.

The data provided in the product_sales_dataset.csv serves as an empirical validation of these concepts, illustrating how categorical and numerical data must be meticulously organized to support high-speed retrieval and optimal distribution across major urban centers. As Swift-Load continues to expand, these computational principles will remain the bedrock of its logistical operations.

Works cited

1. Abstract Data Types - GeeksforGeeks, accessed May 8, 2026, https://www.geeksforgeeks.org/dsa/abstract-data-types/
2. 1.5. Why Study Data Structures and Abstract Data Types? - Runestone Academy, accessed May 8, 2026, https://runestone.academy/ns/books/published/pythonds/Introduction/WhyStudyDataStructuresandAbstractDataTypes.html
3. product_sales_dataset.csv
4. COMP 410 ADT Axiomatic Semantics, accessed May 8, 2026, https://www.cs.unc.edu/~stotts/COMP410/adt/
5. Stacks in C: A Comprehensive Guide to LIFO Data Structures | by ..., accessed May 8, 2026,

https://medium.com/@e_moreira/stacks-in-c-a-comprehensive-guide-to-lifo-data-structures-f246312c938b

6. Introduction to Stack memory - GeeksforGeeks, accessed May 8, 2026, https://www.geeksforgeeks.org/operating-systems/introduction-to-stack-memory/
7. Difference Between Information Hiding and Encapsulation | Baeldung, accessed May 8, 2026, https://www.baeldung.com/java-information-hiding-vs-encapsulation
8. Comp 413 Course Notes - Formal Specification, accessed May 8, 2026, https://h3turing.vmhost.psu.edu/courses/comp413.taw.f98/formal.html
9. accessed May 8, 2026, https://www.jot.fm/issues/issue_2003_01/column2/
10. Encapsulation and Information Hiding, accessed May 8, 2026, http://web.cs.wpi.edu/~cs2102/b16/Lectures/encapsulation.html
11. Advantages of the Information Hiding Principle - TIMOR Programming, accessed May 8, 2026, https://www.timor-programming.org/advantages-of-the-information-hiding-principle.html
12. Imperative Abstract Data Types as the Foundation of Object-Oriented Programming | High school final essays Computer science | Docsity, accessed May 8, 2026, https://www.docsity.com/en/docs/tips-for-writing-a-briefing-document/8169996/
13. (PDF) Abstract Behavior Types: A Foundation Model for Components and Their Composition, accessed May 8, 2026, https://www.researchgate.net/publication/222173382_Abstract_Behavior_Types_A_Foundation_Model_for_Components_and_Their_Composition
14. Support for Object-Oriented Programming | by Prajun Trital | Mar, 2026 | Medium, accessed May 8, 2026, https://medium.com/@prajun_t/support-for-object-oriented-programming-b4907c34315b
15. Lecture 04 - Object-Oriented Concurrent Programming - Universidade de Aveiro › SWEET, accessed May 8, 2026, https://sweet.ua.pt/mos/courses/pcoo/lecture04-print.pdf

16. LIFO, FIFO. The difference between a stack and a queue. A quick guide. - DEV Community, accessed May 8, 2026, https://dev.to/san00/lifo-fifo-the-difference-between-a-stack-and-a-queue-a-quick-guide-1k8m
17. Mastering Stacks and Queues: Understanding LIFO and FIFO Data Structures, accessed May 8, 2026, https://blog.bitsrc.io/mastering-stacks-and-queues-understanding-lifo-and-fifo-data-structures-531c8d17194c
18. Stack Data Structure - Operations, Applications, Implementation - Masai School, accessed May 8, 2026, https://www.masaischool.com/blog/stack-data-structure-explained-with-examples/
19. Push and Pop in Stacks and Queues - CodeChef, accessed May 8, 2026, https://www.codechef.com/learn/course/stacks-and-queues/LSTACKS/problems/STACK11
20. Call Stack and Stack Frames - Medium, accessed May 8, 2026, https://medium.com/@CyberGee/call-stack-and-stack-frames-f1630fd3c781
21. accessed May 8, 2026, https://dev.to/abstractmusa/understanding-stack-operations-how-programs-store-and-release-data-in-memory-48gg#:~:text=Function%20Calls%3A,is%20pushed%20onto%20the%20stack.
22. Understand stack memory management | Chris Bao's Blog, accessed May 8, 2026, https://organicprogrammer.com/2020/08/19/stack-frame/
23. Call stack - Wikipedia, accessed May 8, 2026, https://en.wikipedia.org/wiki/Call_stack
24. Understanding Stack Operations: How Programs Store and Release Data in Memory, accessed May 8, 2026, https://dev.to/abstractmusa/understanding-stack-operations-how-programs-store-and-release-data-in-memory-48gg
25. Stack (abstract data type) - Wikipedia, accessed May 8, 2026, https://en.wikipedia.org/wiki/Stack_(abstract_data_type)

26. axiomatic semantics, accessed May 8, 2026, https://xlinux.nist.gov/dads/HTML/axiomaticSemantics.html
27. Algebraic Specifications for STACK, accessed May 8, 2026, https://www.cs.unc.edu/~stotts/723/senotes/qs4.html
28. LIFO Principle in Stack - GeeksforGeeks, accessed May 8, 2026, https://www.geeksforgeeks.org/dsa/lifo-principle-in-stack/
29. Comparing Time Complexity: Bubble Sort vs. Quick Sort in C++ | by Md Sayeed Firoz, accessed May 8, 2026, https://medium.com/@firozkaif27/comparing-time-complexity-bubble-sort-vs-quick-sort-in-c-2c2ff59a4b4c
30. Analysis of different sorting techniques - GeeksforGeeks, accessed May 8, 2026, https://www.geeksforgeeks.org/dsa/analysis-of-different-sorting-techniques/
31. Why bubble sort is faster than quick sort - Stack Overflow, accessed May 8, 2026, https://stackoverflow.com/questions/46786327/why-bubble-sort-is-faster-than-quick-sort
32. Top 3 search algorithms and their time complexity | by Shreyans Bhansali | Medium, accessed May 8, 2026, https://assistiv-ai.medium.com/top-3-search-algorithms-and-their-time-complexity-e9d619fb7d27
33. Analysis of Algorithms - Stanford, accessed May 8, 2026, http://www-cs-students.stanford.edu/~rashmi/projects/Sorting.pdf
34. Optimizing Urban Transportation Networks: Comparative Analysis of Dijkstra's Algorithm in Graph-Based Shortest Path Solutions - Informatika, accessed May 8, 2026, https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2024-2025/Makalah/Makalah-IF1220-Matdis-2024%20(119).pdf
35. Dijkstra's algorithm - Wikipedia, accessed May 8, 2026, https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
36. A* vs Dijkstra: Choosing the Right Pathfinding Algorithm for EVE ..., accessed May 8, 2026, https://ef-map.com/blog/astar-vs-dijkstra-pathfinding-comparison

37. A* Search and Dijkstra's Algorithm: A Comparative Analysis, accessed May 8, 2026, https://cse442-17f.github.io/A-Star-Search-and-Dijkstras-Algorithm/
38. Comparative Analysis of Dijkstra and A* Algorithms for Determining the Shortest Route - Komunitas Dosen Indonesia, accessed May 8, 2026, https://jurnal.kdi.or.id/index.php/bt/article/download/3474/1795
39. Performance Comparison of A and Dijkstra Algorithms with Bézier Curve in 2D Grid and OpenStreetMap Scenarios, accessed May 8, 2026, https://ir.uitm.edu.my/124627/1/124627.pdf
40. 6 Types of Search Algorithms & How They Work - Luigi's Box, accessed May 8, 2026, https://www.luigisbox.com/blog/types-of-search-algorithms/
41. Space–time tradeoff - Wikipedia, accessed May 8, 2026, https://en.wikipedia.org/wiki/Space%E2%80%93time_tradeoff
42. Memory Usage vs. Speed Definition for Data Structures |... - Fiveable, accessed May 8, 2026, https://fiveable.me/data-structures/key-terms/memory-usage-vs-speed
43. Time-Space Tradeoff: Data Structure Speed vs. Memory - Prepp, accessed May 8, 2026, https://prepp.in/question/which-data-structure-often-results-in-a-time-space-663a4e250368feeaa5c299d2
44. Technique: Space-Time Tradeoff, accessed May 8, 2026, https://cusack.hope.edu/Algorithms/Content/Techniques/Space-Time%20Tradeoff.html?path=Problems/Foundational/Sorting
45. AVL tree - Wikipedia, accessed May 8, 2026, https://en.wikipedia.org/wiki/AVL_tree
46. A Comparative Analysis of Self-Balancing Binary Search Trees 1. AVL Tree, accessed May 8, 2026, https://www.math.nagoya-u.ac.jp/~richard/teaching/s2024/SML_Liu_2.pdf
47. Complexity of different operations in Binary tree, Binary Search Tree and AVL tree, accessed May 8, 2026, https://www.geeksforgeeks.org/dsa/complexity-different-operations-binary-tree-binary-search-tree-avl-tree/

48. Comparing Implementations of Optimal Binary Search Trees - Tufts University, accessed May 8, 2026, https://www.eecs.tufts.edu/~aloupis/comp150/projects/Cori_Jacoby_Alex_King_Final_Report_v2.pdf
49. Stack in Data Structure: Examples, Uses, Implementation, More - WsCube Tech, accessed May 8, 2026, https://www.wscubetech.com/resources/dsa/stack-data-structure
50. The Importance of Data Structures in Software Development - Scribd, accessed May 8, 2026, https://www.scribd.com/document/986171084/The-Importance-of-Data-Structures-in-Software-Development
51. Why Data Structures Matter in Efficient Programming - CelerData, accessed May 8, 2026, https://celerdata.com/glossary/why-data-structures-matter-in-efficient-programming
52. Understanding Data Independence in DBMS: Importance and Benefits - Coding Age, accessed May 8, 2026, https://www.codingage.biz/understanding-data-independence-in-dbms-importance-and-benefits
53. Data Abstraction and Data Independence - GeeksforGeeks, accessed May 8, 2026, https://www.geeksforgeeks.org/dbms/data-abstraction-and-data-independence/